ML Ops (CSCR4205)

B.Tech. (CSE) 3rd YEAR

Semester-VI

Student Names: **Sumit Shrestha**

System ID: **2023850898**

Section: CSH(G2)

Submitted to:

Mr. Nishant Upadhyay

School of Engineering & Technology

Department of Computer Science & Engineering

1. Project Overview

This repository contains solutions to the MLOps Practice Exercise Questions covering Git version control, Python ML workflows, project structuring, and reproducibility. The exercises are designed to give hands-on experience with fundamental MLOps concepts that are essential for managing machine learning projects in a professional environment.

2. Project Structure

The repository follows an MLOps-friendly folder structure for clean separation of data, source code, models, and documentation.

```
mlops-unit1/
├── Exercise 1/
│   ├── data/
│   │   └── sample_data.csv
│   └── dataset_stats.py
├── Exercise 2/
│   ├── data/
│   │   └── data.csv
│   ├── model/
│   │   └── model.pkl
│   ├── src/
│   │   └── main.py
│   └── requirements.txt
└── README.md
```

3. Exercise 1 — Git & Version Control (MLOps Basics)

The following tasks were completed as part of Exercise 1:

1. Created a Git repository for the ML project named mlops-unit1.
2. Wrote a Python script (dataset_stats.py) that loads a dataset and prints basic statistics (min, max, mean) for numeric columns.
3. Committed the code with appropriate commit messages.
4. Created a branch called experiment-v1 and made changes in the code.
5. Merged the branch back into the main branch.

dataset_stats.py — Source Code

```
import csv
from pathlib import Path
from statistics import mean


def load_dataset(file_path: Path) -> list[dict[str, str]]:
    with file_path.open(mode="r", newline="", encoding="utf-8") as csv_file:
        reader = csv.DictReader(csv_file)
        return list(reader)


def to_float_list(rows: list[dict[str, str]], column: str) -> list[float]:
    values: list[float] = []
    for row in rows:
        raw_value = row.get(column, "").strip()
        if not raw_value:
            continue
        values.append(float(raw_value))
    return values


def main() -> None:
    dataset_path = Path(__file__).parent / "data" / "sample_data.csv"
    rows = load_dataset(dataset_path)

    print(f"Dataset loaded from: {dataset_path}")
    print(f"Rows: {len(rows)}")
    print(f"Columns: {len(rows[0]) if rows else 0}")

    if not rows:
        print("No data found.")
        return

    for numeric_column in ("age", "score"):
        values = to_float_list(rows, numeric_column)
        if not values:
            continue
        print(f"\n{numeric_column} statistics:")
        print(f"  min: {min(values):.2f}")
```

```
        print(f"  max: {max(values):.2f}")
        print(f"  mean: {mean(values):.2f}")

    print("\nProcessed numeric columns: age, score")
    print(f"Total records processed: {len(rows)}")


if __name__ == "__main__":
    main()
```

Sample Dataset (sample_data.csv)

name	age	score
Alice	25	88
Bob	30	92
Charlie	35	79
Diana	28	85
Evan	22	90

```
Rows: 5
Columns: 3

age statistics:
  min: 22.00
  max: 35.00
  mean: 28.00

score statistics:
  min: 79.00
  max: 92.00
  mean: 86.80

Processed numeric columns: age, score
Total records processed: 5
```

Fig 1: Output

4. Exercise 2 — Python ML Workflow

A complete ML workflow was implemented in Exercise 2/src/main.py covering data loading, splitting, training, evaluation, and model persistence.

- **Data Loading —** Loads the dataset from data/data.csv using pandas.
- **Data Splitting —** Splits data into training (60%) and testing (40%) sets using train_test_split with random_state=1.
- **Model Training —** Trains a Logistic Regression model on features f1 and f2.
- **Evaluation —** Prints model accuracy using sklearn.metrics.accuracy_score.
- **Model Saving —** Saves the trained model as model/model.pkl using joblib, then loads and verifies predictions from the saved model.

main.py — Source Code

```
# Data Loading using pandas
from pathlib import Path
import pandas as pd

base_dir = Path(__file__).resolve().parent
data_path = base_dir.parent / 'data' / 'data.csv'
model_dir = base_dir.parent / 'model'
model_path = model_dir / 'model.pkl'

data = pd.read_csv(data_path)
df = pd.DataFrame(data)
print("Data Loaded: ")
print(df)

# Splitting data into training and testing
from sklearn.model_selection import train_test_split
x = df[['f1', 'f2']]
y = df['output']
x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.4, random_state=1
)

# Training Logistic Regression model
from sklearn.linear_model import LogisticRegression
logReg = LogisticRegression()
logReg.fit(x_train, y_train)

y_predict = logReg.predict(x_test)
```

```
# Printing model Accuracy
from sklearn import metrics
print("Model Accuracy: ", metrics.accuracy_score(y_test, y_predict))

# Saving trained model using joblib
from joblib import Parallel, delayed
import joblib

model_dir.mkdir(parents=True, exist_ok=True)
joblib.dump(logReg, model_path)
model = joblib.load(model_path)
print("Prediction using saved model: ", model.predict(x_test))
```

Dataset (data.csv)

f1	f2	output
1	10	1
2	9	1
3	8	0
4	7	0
5	6	1
6	5	0
7	4	0
8	3	1
9	2	0
10	1	1

```
Data Loaded:
   f1  f2  output
0   1  10       1
1   2   9       1
2   3   8       0
3   4   7       0
4   5   6       1
5   6   5       0
6   7   4       0
7   8   3       1
8   9   2       0
9  10   1       1
Model Accuracy:  0.5
predection using saved model:  [1 0 0 1]
```

Fig 2: Output

5. Exercise 3 — Dev Workflow & Project Structure

The project is organized following an MLOps-friendly folder structure that cleanly separates data, source code, models, and configuration.

Folder / File	Purpose
data/	Contains datasets
Exercise 2/src/	Source code for ML workflow
Exercise 2/model/	Saved trained models
Exercise 2/requirements.txt	Python dependencies
README.md	Project documentation

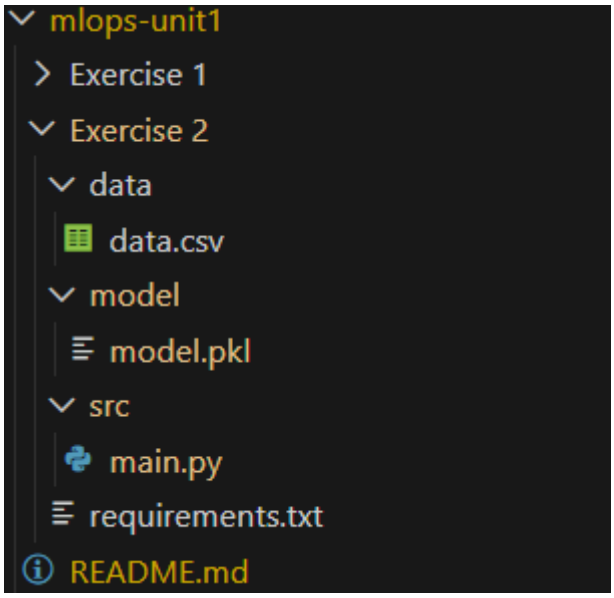

Fig: Project Structure

6. Exercise 4 — Reproducibility Using Python & Git

Python Dependencies
All dependencies are listed in Exercise 2/requirements.txt to ensure anyone can recreate the exact same environment:

joblib==1.5.3
pandas==3.0.1
scikit_learn==1.8.0

How to Recreate the Environment

6. Clone the repository:
   ```
   git clone <repository-url>
   cd mlops-unit1
   ```
7. Create and activate a virtual environment:
   ```
   python -m venv venv
   venv\Scripts\activate        (Windows)
   source venv/bin/activate    (Linux/macOS)
   ```
8. Install dependencies:
   ```
   pip install -r "Exercise 2/requirements.txt"
   ```
9. Run the dataset statistics script (Exercise 1):
   ```
   python dataset_stats.py
   ```
10. Run the ML workflow (Exercise 2):
    ```
    python "Exercise 2/src/main.py"
    ```

7. Technologies Used

- Python 3.12
- pandas — data manipulation and analysis
- scikit-learn — machine learning (Logistic Regression, train/test split, metrics)
- joblib — model serialization / persistence
- Git — version control